Department of Electrical & Computer Engineering
AIR UNIVERSITY

PROJECT REPORT

SIGNALS AND SYSTEMS

Instructor Name

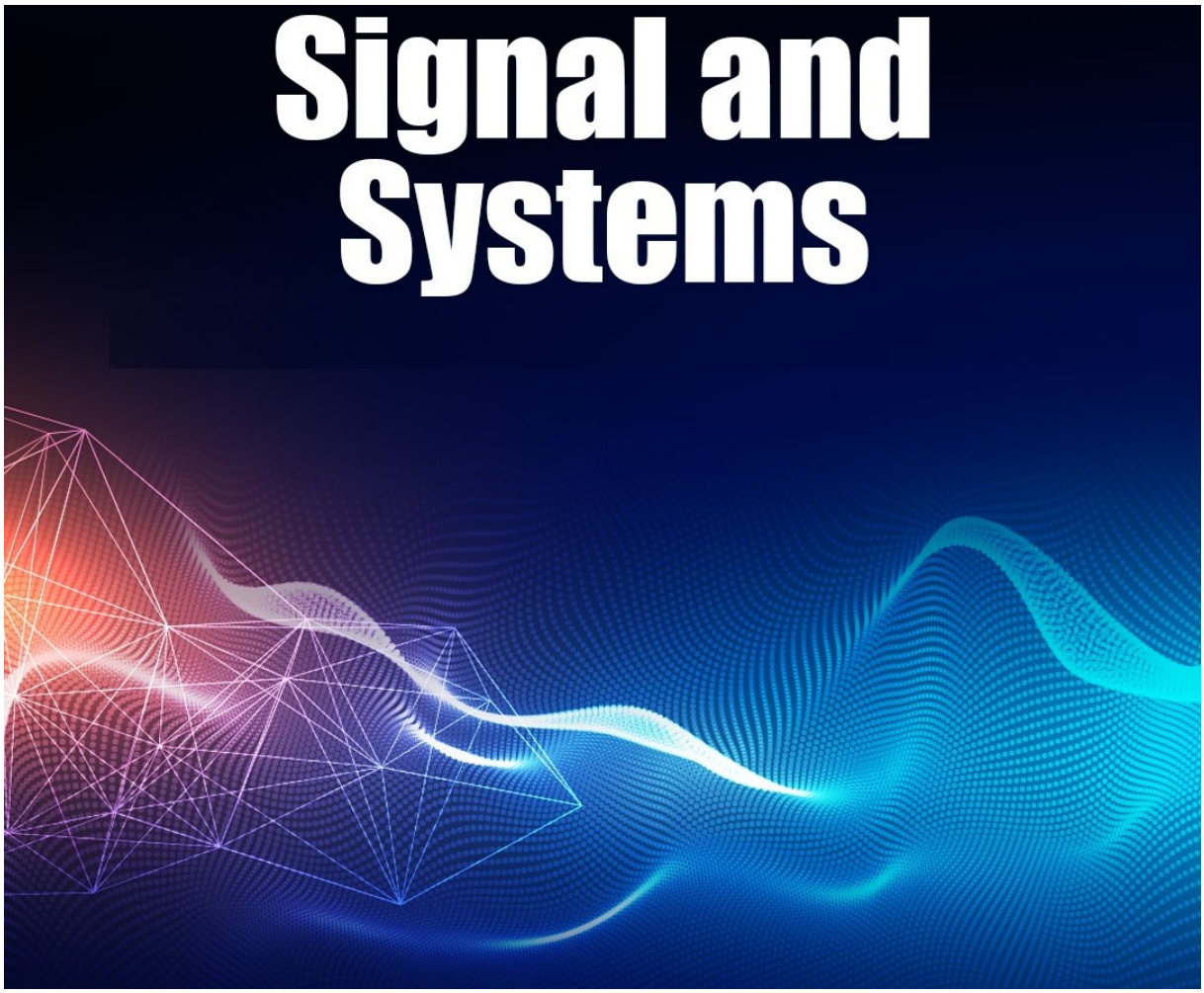
Sir Hafiz Muhammad Talha

Submitted by

Muhammad Burhan Ahmed
Zafir Asad

17/12/2023

Complex Engineering Problem



Interactive Signal Plotter using Matlab GUI

ABSTRACT

This project presents a MATLAB-based Graphical User Interface (GUI) application for signal processing tasks. The GUI is designed using Matlab's App Designer and incorporates functionalities for visualizing and manipulating signals in the continuous time domain. The primary features include the display of Sin waves and Rectangular signals with customized amplitude and frequency, manual time shifting for signals, convolution of two signals with the option to consider one as an impulse response, and the plotting of Fourier series coefficients for a user-selected signal.

The application's user interface comprises interactive elements such as axes for signal plots, sliders and edit fields for parameter input (amplitude, frequency, time shift), and buttons to trigger specific actions (convolution, Fourier series coefficients). Users can dynamically interact with the GUI to observe the effects of parameter changes on signal characteristics.

The implementation involves utilizing Matlab's built-in functions for signal processing tasks, including convolution and Fourier series coefficient calculations. The GUI enables users to explore and analyze the behavior of signals, providing a user-friendly platform for signal processing experimentation and learning.

Contents

Problem Statement

The primary goal of this project is to design signal processing GUI, including displaying signals, allowing user input for parameters, performing convolution, plotting Fourier series coefficients, and ensuring the signals are continuous-time and properly scaled for visualization.

OBJECTIVES

- Hands-on experience in designing and configuration.
- How to troubleshoot a Matlab GUI.
- Familiarization with Matlab.
- This problem involves Interdependence and is a high-level problem including many components parts and sub-problems.

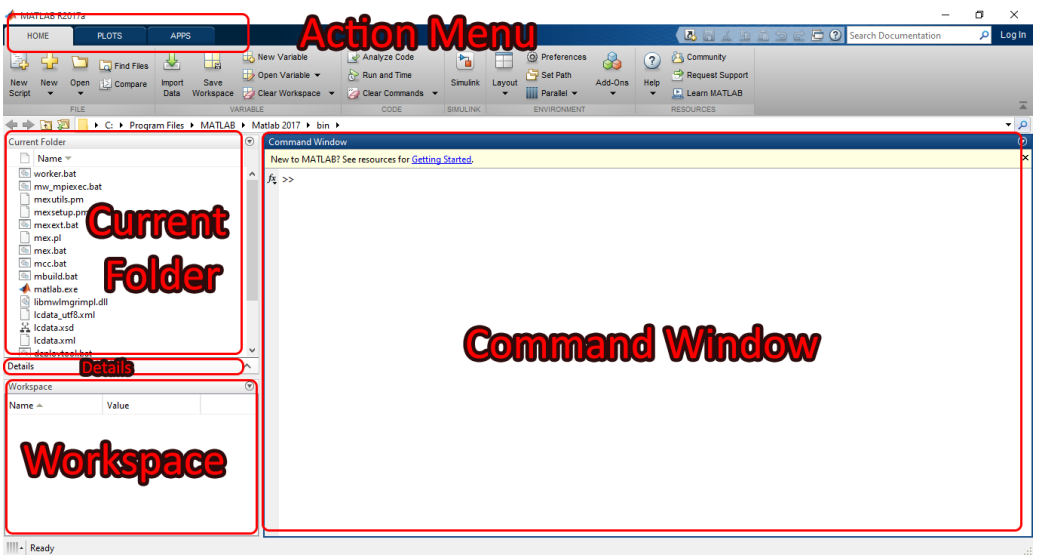
Software Used :

- Matlab 2017a

1 INTRODUCTION

1.1 Matlab 2017a

MATLAB (Matrix Laboratory) is a high-performance programming language and environment primarily used for numerical computing, data analysis, and visualization. It is useful as it provides a comprehensive environment for numerical computing, data analysis, and visualization. Its wide range of built-in functions, tool boxes, and scripting capabilities make it a versatile tool for engineers, scientists, and researchers across various domains. I used MATLAB R2017a. If you are using a different version, the interface may be different. The following components are important to be familiarized with:



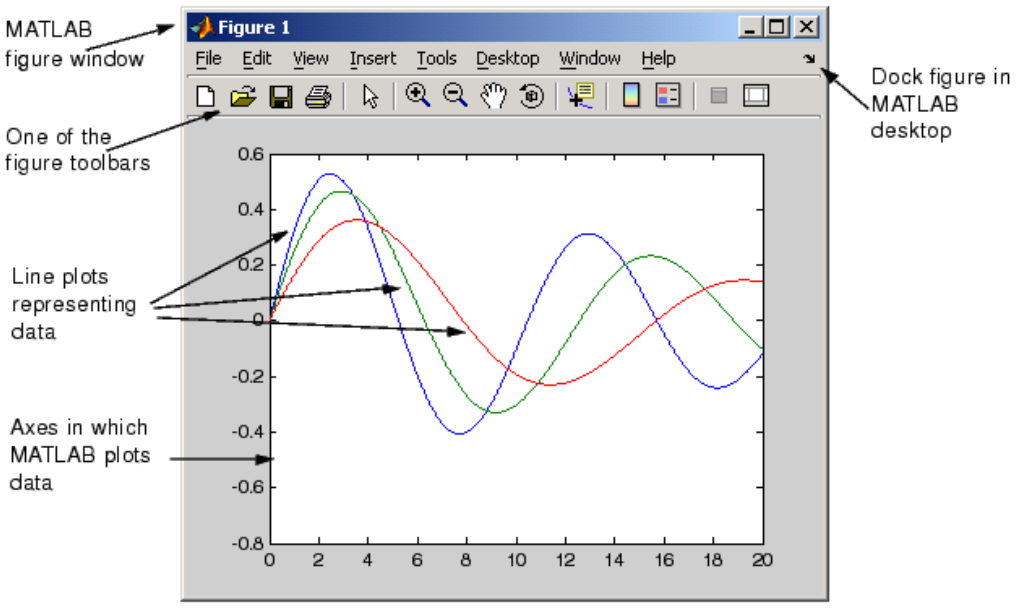
1.1.1 User Applications

It is widely used in various scientific, engineering, and academic fields. Here are some key purposes and applications of MATLAB

1. Numerical Computing (Problems involving matrix operations, linear algebra, and differential equations.)
2. Data Analysis and Visualization
3. Signal and Image Processing
4. Control System Design
5. Machine Learning and Statistics
6. Simulation and Modeling
7. Communication and Signal Processing
8. Mathematical and Scientific Research

1.1.2 Graphics and Graphical User Interface

MATLAB provides a variety of functions for creating 2D and 3D plots, charts, and visualizations.



Some commonly used functions include:

1.1.3 Plots & Charts in Matlab

Plot: For creating 2D line plots.

```
x = linspace(0, 2*pi, 100);
y = sin(x);
plot(x, y);
```

Axes: For creating plots in GUI.

```
x = rand(1, 100);
y = rand(1, 100);
plot(handles.third, x, y);
```

1.1.4 Guide Environment.

MATLAB provides a GUIDE (Graphical User Interface Development Environment) for creating GUIs. You can design and create GUIs using the drag-and-drop interface provided by GUIDE. Following are some components that must be understood while working with Matlab GUI

- Open GUIDE: You can open GUIDE by typing guide in the MATLAB command window.
- Design: Use the GUI design tools to create your user interface. You can add buttons, sliders, text boxes, plots, and other components.
- Callback Functions: Attach MATLAB code (callback functions) to GUI components to specify what should happen when a user interacts with them.
- Generate Code: GUIDE allows you to generate MATLAB code for your GUI. You can then modify this code if needed.

1.2 Signals

A signal is a function that conveys information about a phenomenon. Signals can be categorized into different types but we are only concerned about continuous-time signals which is as per project requirements.

1.2.1 Continuous Time Signal

Continuous-time signals are defined for all values of time in a specified interval. An example is an analog signal, like the sin wave which is also a periodic signal.

1.3 Sin Wave

A sine wave is a smooth, continuous, and periodic waveform that oscillates regularly between a maximum and minimum value following the trigonometric function sine.

Calculation:

Following formula was used to generate sin wave in Matlab;

$$y(t) = A \cdot \sin(2\pi f t) \tag{1}$$

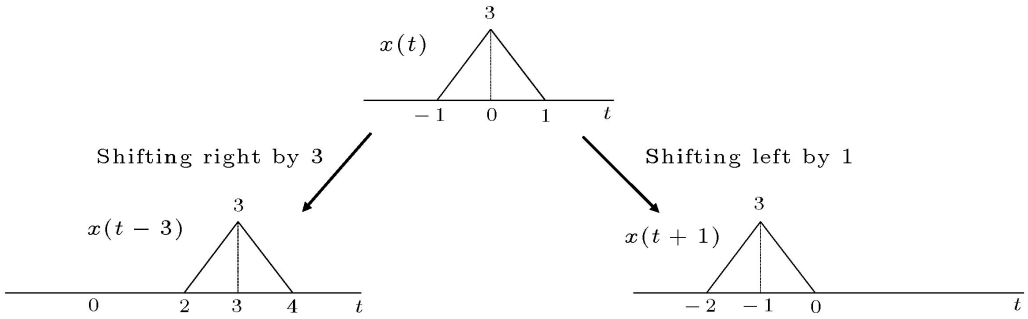
1.4 Rectangular Wave

A rectangular wave, also known as a square wave, is a periodic waveform that alternates between two distinct levels, typically a high and a low value, forming a square-shaped pattern.
Calculation:
Following formula was used to generate Rectangular wave in Matlab;

$$y(t) = A \cdot square(2\pi ft, duty_cycle) \tag{2}$$

1.5 Operations on Signals

Operations on signals are fundamental in the field of signal processing. Signals can represent various physical phenomena such as audio, images, sensor readings, and more. Here are some common operations performed on signals:



- Left Shifting

$$y(t) = x(t + t_o) \tag{3}$$

- Right Shifting

$$y(t) = x(t - t_o) \tag{4}$$

- Convolution

$$y(t) = \int_{-\infty}^{\infty} x_1(\tau) \cdot x_2(t - \tau)\, d\tau \tag{5}$$

1.6 Convolution of systems

Convolution is a mathematical operation that combines two signals to produce a third signal, expressing the amount of overlap between them. The convolution operation is widely used in signal processing, image processing, and other fields. Let's delve into some additional aspects of convolution

1.6.1 Properties of Convolution

Mathematical properties of convolution are given below:

- Commutative $x(t) * h(t) = h(t) * x(t)$
- Associative $x(t) * h_1(t) * h_2(t) = x(t) * h_1(t) * h_2(t)$
- Distributive $x(t) * h_1(t) + h_2(t) = x(t) * h_1(t) + x(t) * h_2(t)$

1.7 Fourier Transform

The Fourier series represents a periodic function as a sum of sine and cosine functions (or complex exponential) with different frequencies and amplitudes. For a periodic function $f(t)$ with period T, its Fourier series in the continuous domain can be expressed as:

$$f(t) = a_0 + \sum_{n=1}^{\infty} [a_k \cos(kw_o t) + b_k \sin(k2\pi nt)] \tag{6}$$

where a_0,a_k and b_k are Coefficients and can be calculated using the following formulas:
DC Component:

$$a_0 = \frac{1}{T_o} \int_T f(t)\, dt \tag{7}$$

$$a_k = \frac{2}{T_o} \int_{T_o} f(t) \cos(kw_o t)\ dt \tag{8}$$

$$b_k = \frac{2}{T_o} \int_{T_o} f(t) \sin(kw_o t)\ dt \tag{9}$$

2 Deliverables

Following are the functionalities provided by project

- Time Shifting of given Signal (Left & Right)
- Plotting Desired Signal (Sin wave & Rectangular wave)
- Convolution of Signals
- Fourier Transform of Signal

3 Procedure

Here are the steps taken to complete this GUI design in Matlab:

1. Open Matlab2017a and create a new project.
2. Setting up the environment, type guide in the command window and save your new .fig file.
3. Design the layout of GUI. Drag and drop the required components from the components palette to create the UI elements for signals, input parameters, time shift, convolution and Fourier series coefficients.
4. Starting from the first task Implement the Sin wave and Rectangular signal plots.
5. Use the edit box for amplitude, frequency and duty cycle which will enable the user to get any desired signal plotted on GUI.
6. Allow users to input time shifts manually for both signals. I use checkbox fields for manual input.

7. I Implement a push button callback to perform convolution when clicked. conv function was used to perform convolution. I considered signal 'a' as input and signal 'b' as an impulse response.
8. Similarly, a button callback to plot the Fourier series coefficients for a signal of my choice.
9. A clear button was also Implemented to clear all the data on the GUI.
10. Time scale and axes are appropriately set for displaying or plotting the signals.
11. Lastly GUI was Tested for its functionalities and necessary adjustments were made as needed to ensure everything works correctly.
12. These steps should help you create the design that meets the requirements of the scenario.

4 Working Methodology

4.1 GUI Design

1. Sin Wave and Rectangular Signal (Functions like plot or stem to plot the wave)
2. Both signals are kept global so that they can easily be used in other functions to perform required tasks.
3. The Callback function is used to perform convolution via the conv function and plots the result.
4. Functions like xlim and ylim are used to set the axis limits.
5. Similarly, a Callback function is used to plot Fourier series coefficients for a signal of your choice.

5 Source Code

```
function varargout = main(varargin)
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @main_OpeningFcn, ...
                   'gui_OutputFcn',  @main_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
```

```
    gui_mainfcn(gui_State, varargin{:});
end

function main_OpeningFcn(hObject, eventdata, handles, varargin)

handles.output = hObject;

guidata(hObject, handles);

function varargout = main_OutputFcn(hObject, eventdata, handles)

varargout{1} = handles.output;

function amp_Callback(hObject, eventdata, handles)

function amp_CreateFcn(hObject, eventdata, handles)

if ispc && isequal(get(hObject,'BackgroundColor'),
   get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end

function freq_Callback(hObject, eventdata, handles)

function freq_CreateFcn(hObject, eventdata, handles)

if ispc && isequal(get(hObject,'BackgroundColor'),
   get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end

function plot_Callback(hObject, eventdata, handles)

global selected
global t
global sinSignal
global squareWave
global t_rec
    amplitude = str2double(get(handles.amp, 'String'));
    frequency = str2double(get(handles.freq, 'String'));
    t =linspace(0,5,1000);
    %t = linspace(0, 1,1000); % Time vector

if strcmp(selected, 'Sin wave')
    % Generate sin wave
    sinSignal = amplitude * sin(2 * pi * frequency * t);

    % Plot sin wave
    plot(handles.prime, t, sinSignal,'r','LineWidth', 2);
    xlabel(handles.prime, 'Time');
    ylabel(handles.prime, 'Amplitude');
```

```
    title(handles.prime, 'Sine Wave');
    grid(handles.prime, 'on');
    %axis(handles.prime ,[min(t) max(t) min(sinSignal)-1
       max(sinSignal)+1]);

elseif strcmp(selected, 'Rectangular wave')
    dutyCycle = str2double(get(handles.duty, 'String')); % Convert
       percentage to fraction
    t_rec =linspace(0,5,1000);
    %t_rec = linspace(-20,20);
    squareWave = amplitude * square(2 * pi * (frequency) * t_rec,
       dutyCycle);

    % Plot the square wave
    plot(handles.third, t_rec, squareWave, 'm', 'LineWidth', 2);
    xlabel(handles.third, 'Time');
    ylabel(handles.third, 'Amplitude');
    title(handles.third, 'Rectangular Wave');
    grid(handles.third, 'on');
    %axis(handles.third ,[min(t_rec)-5 max(t_rec)+5 min(squareWave)-5
       max(squareWave)+5]);
end

function menu_Callback(hObject, eventdata, handles)
contents = cellstr(get(hObject, 'String'));
option = contents{get(hObject, 'Value')};

global selected
selected = option;


if strcmp(selected, 'Sin wave')
    set(handles.dc,'Visible','off');
    set(handles.duty, 'Visible', 'off');
elseif strcmp(selected, 'Rectangular wave')
    set(handles.dc, 'Visible', 'on');
    set(handles.duty, 'Visible', 'on');
end

function menu_CreateFcn(hObject, eventdata, handles)
if ispc && isequal(get(hObject,'BackgroundColor'),
   get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end

function menu_KeyPressFcn(hObject, eventdata, handles)

function duty_Callback(hObject, eventdata, handles)

function duty_CreateFcn(hObject, eventdata, handles)
```

```
110 if ispc && isequal(get(hObject,'BackgroundColor'),
       get(0,'defaultUicontrolBackgroundColor'))
111     set(hObject,'BackgroundColor','white');
112 end
113
114 function edit4_Callback(hObject, eventdata, handles)
115
116 function edit4_CreateFcn(hObject, eventdata, handles)
117 if ispc && isequal(get(hObject,'BackgroundColor'),
       get(0,'defaultUicontrolBackgroundColor'))
118     set(hObject,'BackgroundColor','white');
119 end
120
121
122 function fsc_Callback(hObject, eventdata, handles)
123 % hObject    handle to fs (see GCBO)
124 % eventdata  reserved - to be defined in a future version of MATLAB
125 % handles    structure with handles and user data (see GUIDATA)
126 global squareWave
127 % Check the state of the radio button
128 if get(hObject,'Value') == 1 % If the radio button is selected
129     set(handles.con, 'Value', 0);
130     amplitude=2;
131     t = linspace(-10, 10, 100);
132     rect_function = rectpuls(t, amplitude);
133     %Fast Fourier Transform (FFT), fftshift is a function that
          rearranges the outputs of the FFT operation
134     sinc = fftshift(fft(rect_function));
135     % Plot the coefficients
136     plot(handles.four,t, abs(sinc),'b');
137     hold on
138     plot(handles.four,t, abs(sinc),'o','MarkerEdgeColor', 'r');
139     hold off
140     xlabel(handles.four,'Coefficient Index (n)');
141     ylabel(handles.four,'Magnitude');
142     title(handles.four,'Fourier Series Coefficients of a Rectangular
          Wave');
143     grid(handles.four, 'on');
144     legend('Sinc Function', 'Fourier Series Coefficient');
145     %axis([-1 50 -5  15]); % Set x-axis from 0 to 1 and y-axis from
          -1 to 1
146 end
147
148 function clc_Callback(hObject, eventdata, handles)
149 cla(handles.third);
150 cla(handles.second);
151 cla(handles.prime);
152 cla(handles.four);
153
154 set(handles.con, 'Value', 0);
155 set(handles.fsc, 'Value', 0);
```

```
156
157 set(handles.left, 'Value', 0);
158 set(handles.right, 'Value', 0);
159
160 grid(handles.second, 'off');
161 grid(handles.third, 'off');
162 grid(handles.prime, 'off');
163 grid(handles.third, 'off');
164
165 % Clear the values of edit boxes (assuming these are edit boxes)
166 set(handles.shift, 'String', '');
167 set(handles.amp, 'String', '');
168 set(handles.freq, 'String', '');
169 set(handles.duty, 'String', '');
170
171 % Clear titles and labels
172 delete(get(handles.third,'Title'));
173 delete(get(handles.second,'Title'));
174 delete(get(handles.prime,'Title'));
175 delete(get(handles.four,'Title'));
176
177 delete(get(handles.third,'XLabel'));
178 delete(get(handles.second,'XLabel'));
179 delete(get(handles.prime,'XLabel'));
180 delete(get(handles.four,'XLabel'));
181
182 delete(get(handles.third,'YLabel'));
183 delete(get(handles.second,'YLabel'));
184 delete(get(handles.prime,'YLabel'));
185 delete(get(handles.four,'YLabel'));
186
187 % --- Executes during object creation, after setting all properties.
188 function msg_CreateFcn(hObject, eventdata, handles)
189 % hObject    handle to msg (see GCBO)
190 % eventdata  reserved - to be defined in a future version of MATLAB
191 % handles    empty - handles not created until after all CreateFcns
    called
192 msgbox('Welcome to my GUI');
193
194
195 % --- Executes on button press in left.
196 function left_Callback(hObject, eventdata, handles)
197 % hObject    handle to left (see GCBO)
198 % eventdata  reserved - to be defined in a future version of MATLAB
199 % handles    structure with handles and user data (see GUIDATA)
200 if get(hObject,'Value') == 1
201      set(handles.right, 'Value', 0);
202     set(handles.shift, 'Visible', 'on');
203 else
204     set(handles.shift, 'Visible', 'off');
205 end
```

```
206 % Hint: get(hObject,'Value') returns toggle state of left
207
208
209 % --- Executes on button press in right.
210 function right_Callback(hObject, eventdata, handles)
211 % hObject    handle to right (see GCBO)
212 % eventdata  reserved - to be defined in a future version of MATLAB
213 % handles    structure with handles and user data (see GUIDATA)
214 if get(hObject,'Value') == 1
215      set(handles.left, 'Value', 0);
216     set(handles.shift, 'Visible', 'on');
217 else
218     set(handles.shift, 'Visible', 'off');
219 end
220 % Hint: get(hObject,'Value') returns toggle state of right
221
222
223 % --- Executes on selection change in popupmenu3.
224 function popupmenu3_Callback(hObject, eventdata, handles)
225 % hObject    handle to popupmenu3 (see GCBO)
226 % eventdata  reserved - to be defined in a future version of MATLAB
227 % handles    structure with handles and user data (see GUIDATA)
228
229 % Hints: contents = cellstr(get(hObject,'String')) returns popupmenu3
    contents as cell array
230 %        contents{get(hObject,'Value')} returns selected item from
    popupmenu3
231
232
233 % --- Executes during object creation, after setting all properties.
234 function popupmenu3_CreateFcn(hObject, eventdata, handles)
235 % hObject    handle to popupmenu3 (see GCBO)
236 % eventdata  reserved - to be defined in a future version of MATLAB
237 % handles    empty - handles not created until after all CreateFcns
    called
238
239 % Hint: popupmenu controls usually have a white background on Windows.
240 %       See ISPC and COMPUTER.
241 if ispc && isequal(get(hObject,'BackgroundColor'),
    get(0,'defaultUicontrolBackgroundColor'))
242     set(hObject,'BackgroundColor','white');
243 end
244
245
246 % --- Executes on button press in togglebutton1.
247 function togglebutton1_Callback(hObject, eventdata, handles)
248 % hObject    handle to togglebutton1 (see GCBO)
249 % eventdata  reserved - to be defined in a future version of MATLAB
250 % handles    structure with handles and user data (see GUIDATA)
251
252 % Hint: get(hObject,'Value') returns toggle state of togglebutton1
```

```
253
254
255
256 function shift_Callback(hObject, eventdata, handles)
257 % hObject    handle to shift (see GCBO)
258 % eventdata  reserved - to be defined in a future version of MATLAB
259 % handles    structure with handles and user data (see GUIDATA)
260     % Get the value entered in the shift textbox from the GUI
261     global t
262     global t_rec
263     global selected
264     global sinSignal
265     global squareWave
266
267     CT_shift = str2double(get(handles.shift, 'String'));
268
269     if strcmp(selected, 'Sin wave') &&  get(handles.left, 'Value') ==
          1
270     shifted_sine_wave=t-CT_shift;
271     plot(handles.second, shifted_sine_wave, sinSignal,
          'k','LineWidth', 2);
272     xlabel(handles.second, 'Time');
273     ylabel(handles.second, 'Amplitude');
274     title(handles.second, 'Sin Wave with Left Shift');
275     grid(handles.second, 'on');
276     %axis(handles.second ,[min(t)-1 max(t)+1 min(sinSignal)-1
          max(sinSignal)+1]);
277
278     elseif  strcmp(selected, 'Sin wave') && get(handles.right,
          'Value') == 1
279     shifted_sine_wave=t+CT_shift;
280     plot(handles.second, shifted_sine_wave, sinSignal,
          'k','LineWidth', 2);
281     xlabel(handles.second, 'Time');
282     ylabel(handles.second, 'Amplitude');
283     title(handles.second, 'Sin Wave with Right Shift');
284     grid(handles.second, 'on');
285     %axis(handles.second ,[min(t)-1 max(t)+1 min(sinSignal)-1
          max(sinSignal)+1]);
286
287     elseif  strcmp(selected, 'Rectangular wave') && get(handles.left,
          'Value') == 1
288     shifted_rect_wave=t_rec-CT_shift;
289     plot(handles.four, shifted_rect_wave, squareWave,
          'k','LineWidth', 2);
290     xlabel(handles.four, 'Time');
291     ylabel(handles.four, 'Amplitude');
292     title(handles.four, 'Rectangular Wave with Left Shift');
293     grid(handles.four, 'on');
294     %axis(handles.four ,[min(t_rec)-5 max(t_rec)+5 min(squareWave)-1
          max(squareWave)+1]);
```

```
295
296     elseif  strcmp(selected, 'Rectangular wave') &&
           get(handles.right, 'Value') == 1
297     shifted_rect_wave=t_rec+CT_shift;
298     plot(handles.four, shifted_rect_wave, squareWave,
           'k','LineWidth', 2);
299     xlabel(handles.four, 'Time');
300     ylabel(handles.four, 'Amplitude');
301     title(handles.four, 'Rectangular Wave with Right Shift');
302     grid(handles.four, 'on');
303     %axis(handles.four ,[min(t_rec)-5 max(t_rec)+5 min(squareWave)-1
           max(squareWave)+1]);
304     else
305      disp('Please select the wave and shift direction.');
306     end
307
308
309 % Hints: get(hObject,'String') returns contents of shift as text
310 %        str2double(get(hObject,'String')) returns contents of shift
       as a double
311
312
313 % --- Executes during object creation, after setting all properties.
314 function shift_CreateFcn(hObject, eventdata, handles)
315 % hObject    handle to shift (see GCBO)
316 % eventdata  reserved - to be defined in a future version of MATLAB
317 % handles    empty - handles not created until after all CreateFcns
       called
318
319 % Hint: edit controls usually have a white background on Windows.
320 %       See ISPC and COMPUTER.
321 if ispc && isequal(get(hObject,'BackgroundColor'),
       get(0,'defaultUicontrolBackgroundColor'))
322     set(hObject,'BackgroundColor','white');
323 end
324
325
326 % --- Executes on button press in con.
327 function con_Callback(hObject, eventdata, handles)
328 % hObject    handle to con (see GCBO)
329 % eventdata  reserved - to be defined in a future version of MATLAB
330 % handles    structure with handles and user data (see GUIDATA)
331 global sinSignal
332 global squareWave
333 %global t
334
335 if get(hObject,'Value') == 1 % If the radio button is selected
336 set(handles.fsc, 'Value', 0);
337 set(handles.clc, 'Value', 0);
338     y = conv(sinSignal, squareWave);
339     t = 0:(length(y)-1);
```

```
    % Plotting the result
    plot(handles.second, t, y, 'k', 'LineWidth', 2);
    xlabel(handles.second, 'Time');
    ylabel(handles.second, 'Amplitude');
    title(handles.second, 'Convolution');
    grid(handles.second, 'on');
    ylim(handles.second, [min(y)-1 max(y)+1]);
end
% Hint: get(hObject,'Value') returns toggle state of con
```

Listing 1: Project Code

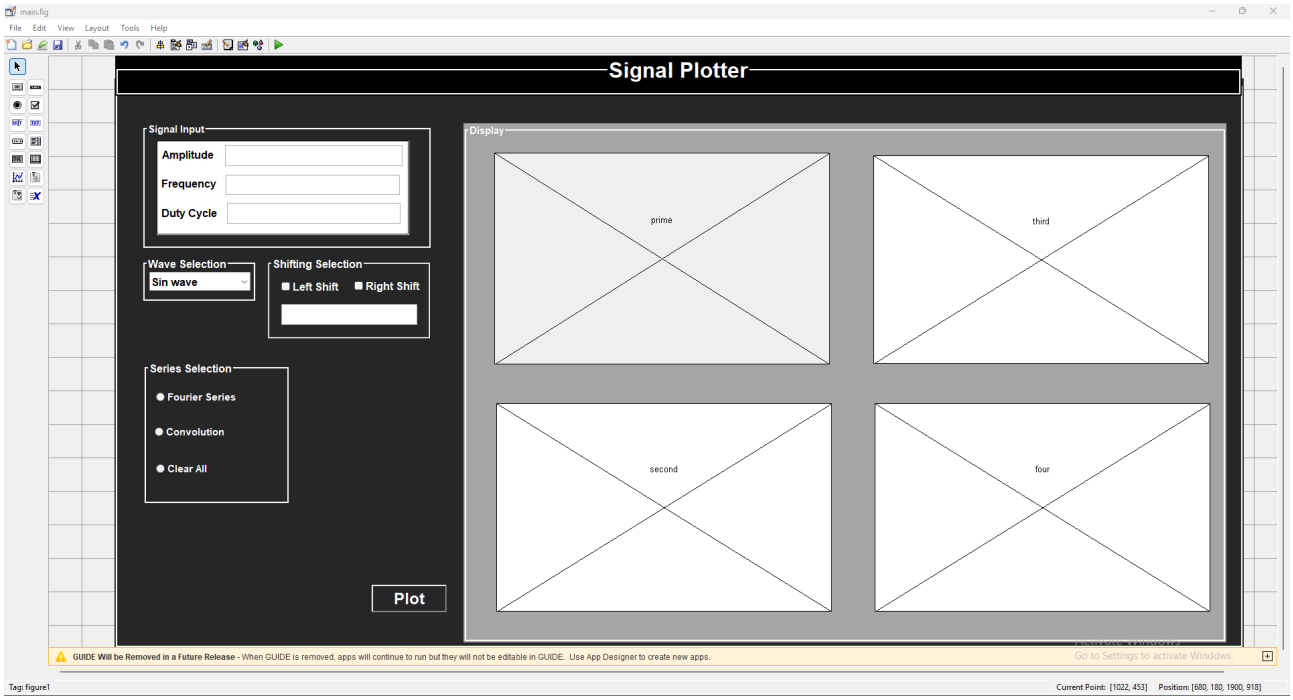
6 Results & Screen Shots

Following are the snaps for Project

Screenshot 1 (Main Panel)

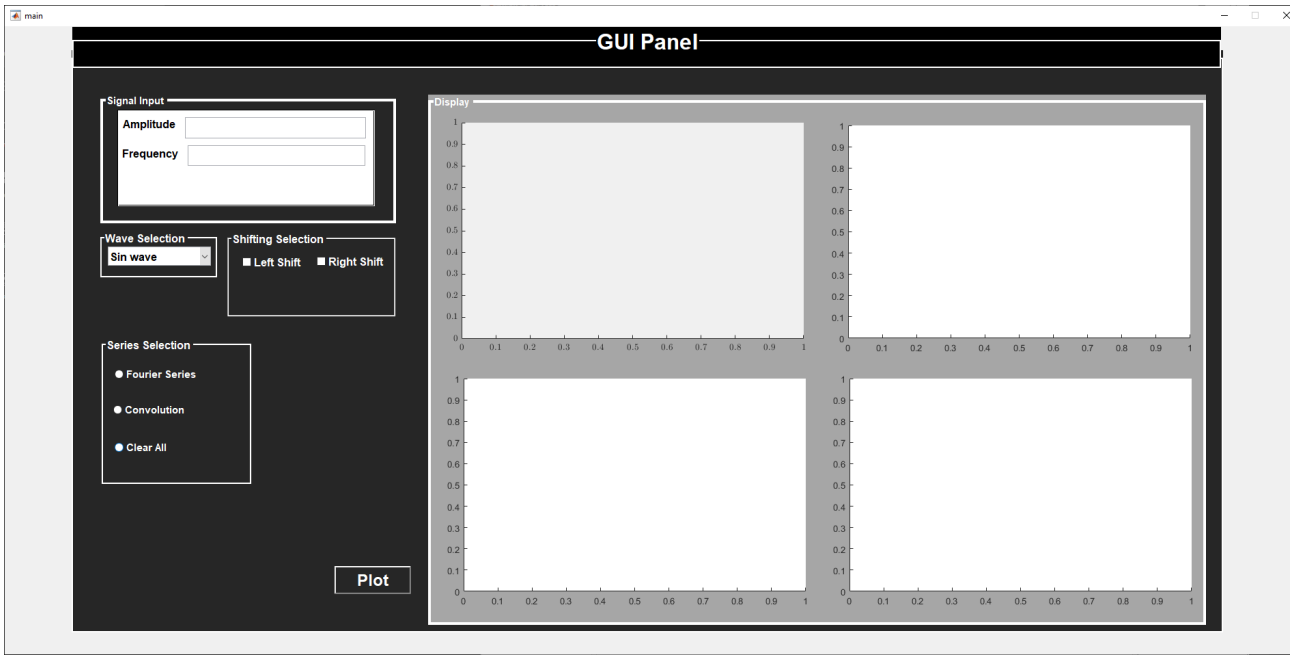
Below figure represents the UI of my GUI Project(in edit) with the following components:

- Main Panel
- Signal Input
- Display Selection
- Shifting Selection
- Wave Selection
- Series Selection



Screenshot 2 (GUI in Run-Time)

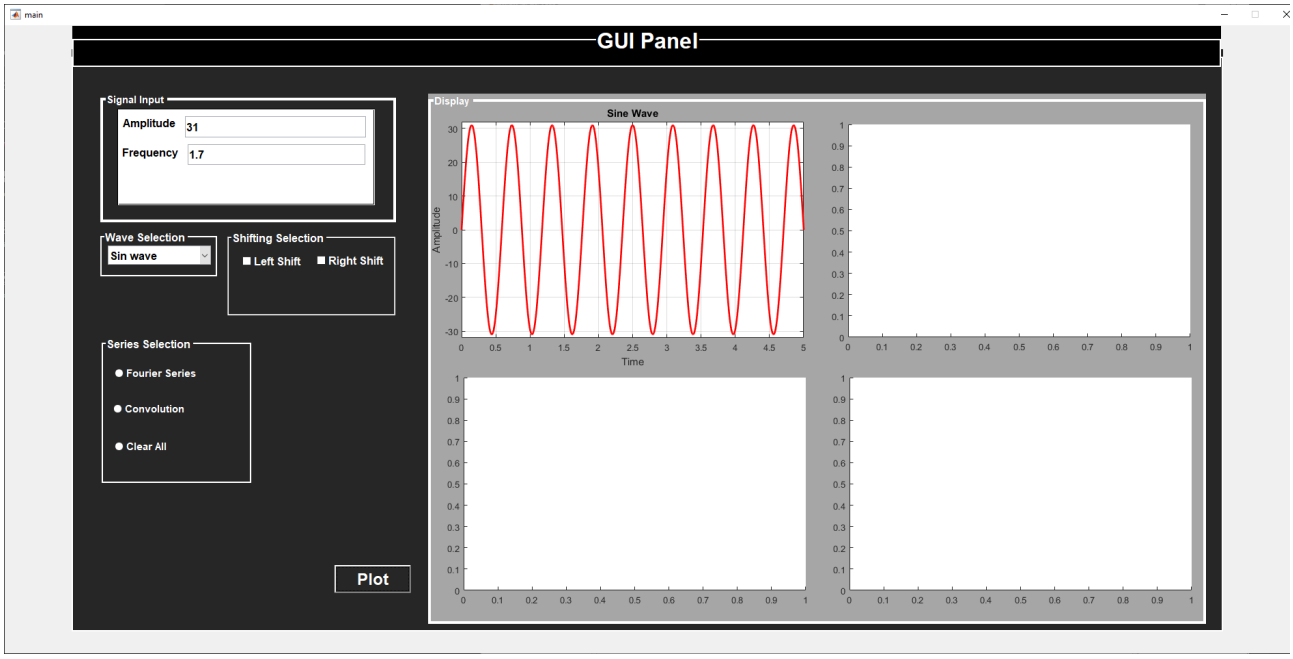
This is the User Interface of my Application which enables the user to view Sin and Rectangular Wave at the same time as well as their shifted output.



Screenshot 3 (Sin Wave)

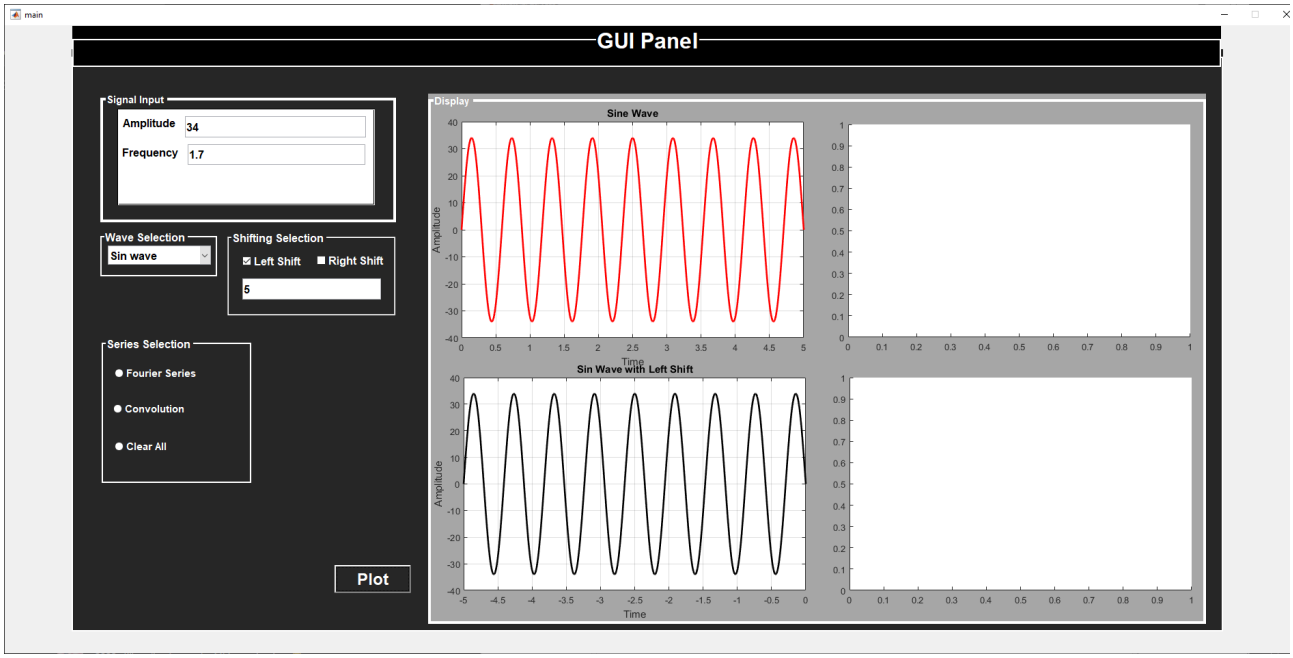
This figure represents a custom Sin Wave plotted by the user with 31 Amplitude and 1.7Hz Frequency. the x-axis represents time and the y-axis represents the Amplitude of the signal with the appropriate title.

Shifting can be represented in two ways, one by making the x-axis fixed and the other by keeping the time axis relative to the signal. In this project, I used the second approach as I am not aware of t



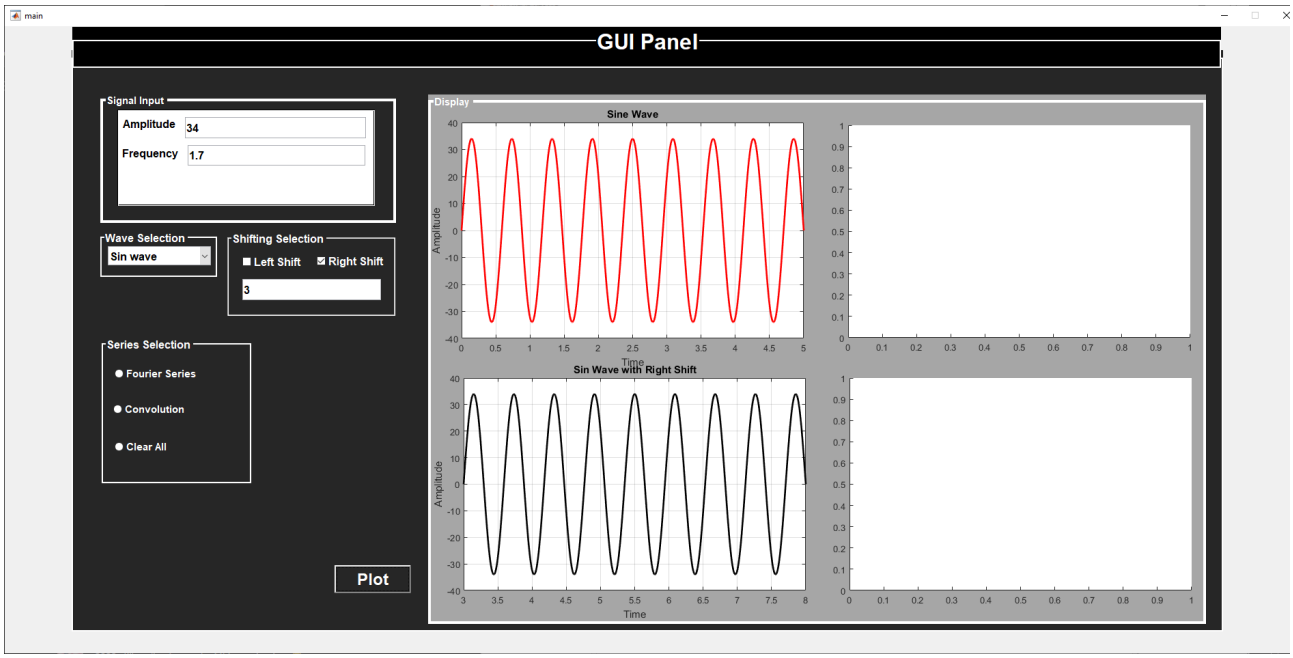
Screenshot 4 (LEFT-SHIFT)

Now I applied a left Shift of 5 to this Sin wave and the resultant signal is plotted below for comparison with the original signal. The solution can be verified by comparing the time-axis of both signals.



Screenshot 5 (RIGHT-SHIFT)

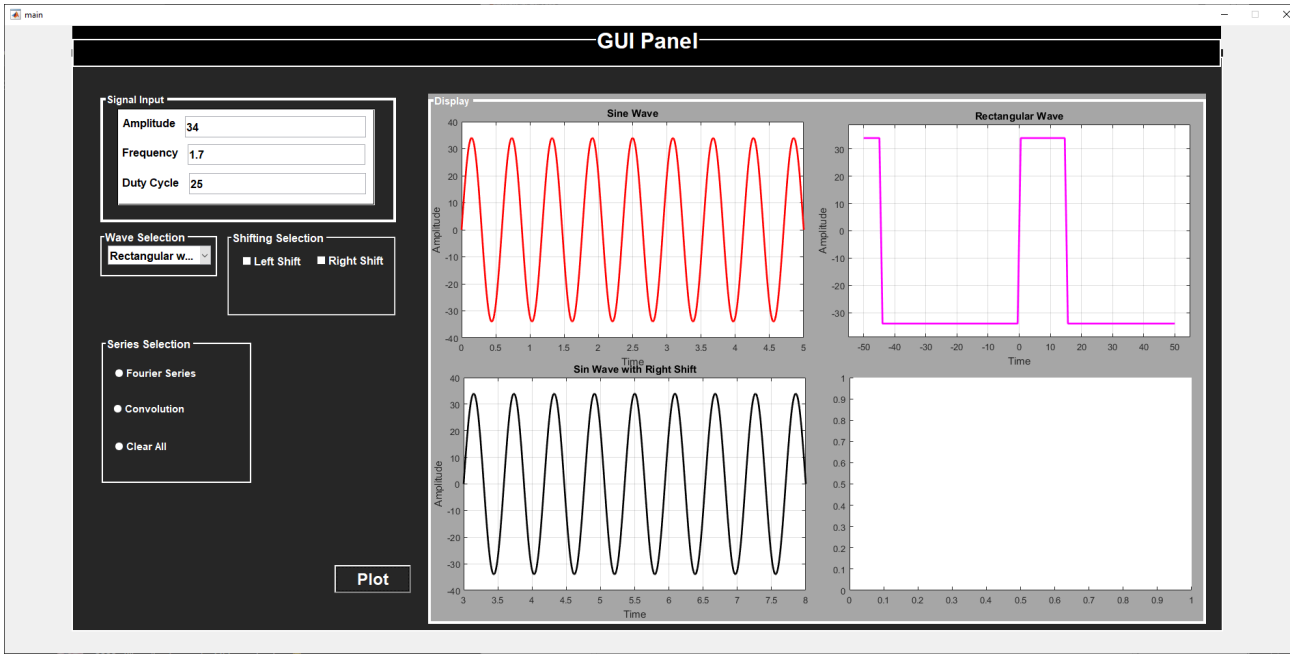
Now I applied a left Shift of 5 to this Sin wave and the resultant signal is plotted below for comparison with the original signal. The solution can be verified by comparing the time-axis of both signals.



Screenshot 6 (Rectangular Wave)

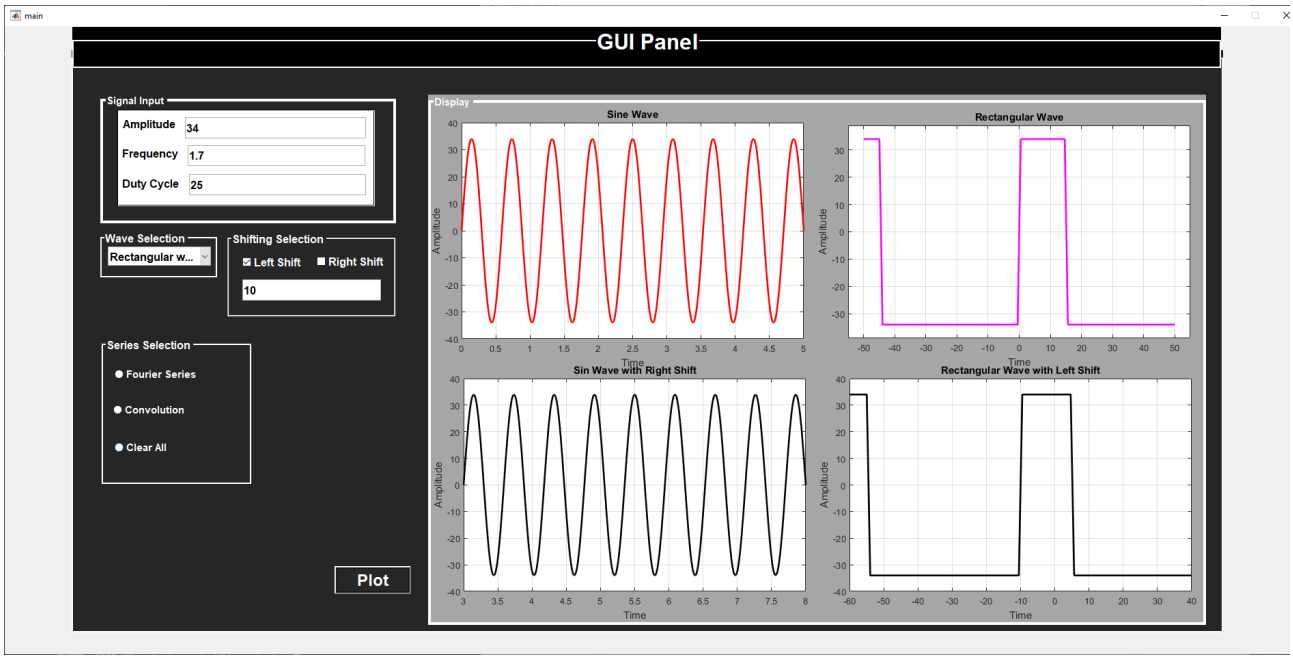
Now I plotted a custom Rectangular Wave with 34 Amplitude and 1.7Hz Frequency with 25% duty cycle. x-axis represents time and the y-axis represents the Amplitude of the signal with

the appropriate title. The plotted wave clearly shows that the signal will remain high for 25% and low for 75%.



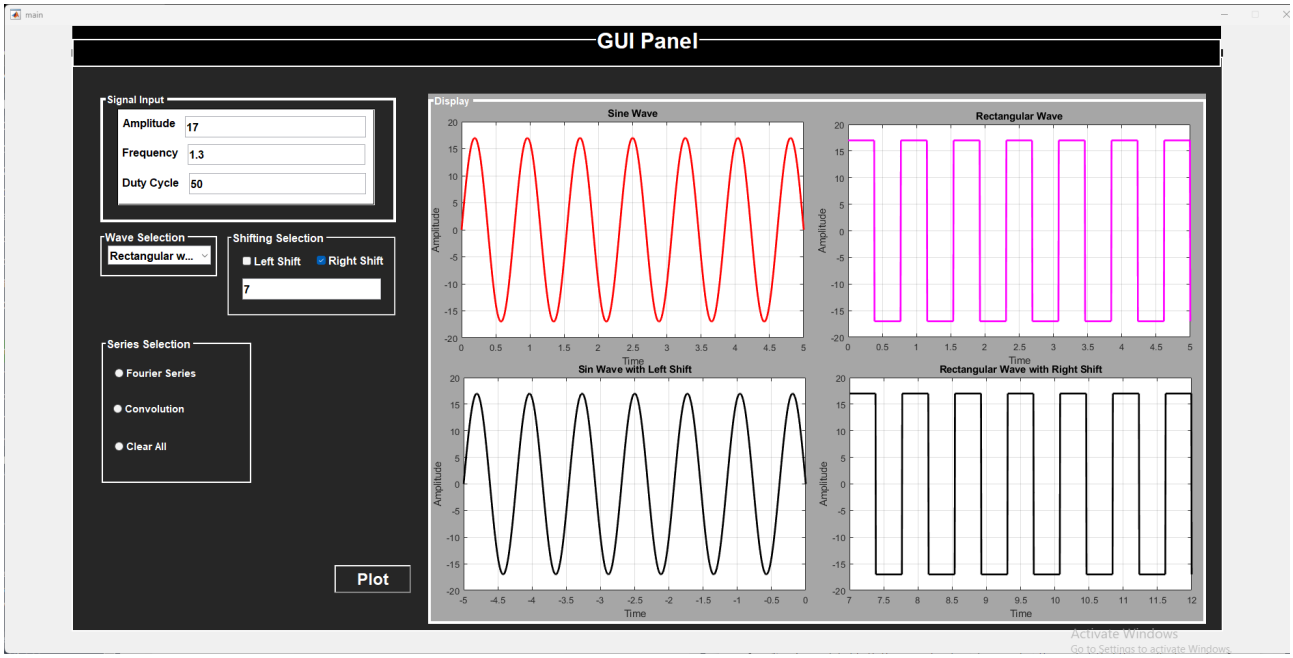
Screenshot 7 (Left-Shift)

Now I applied a left Shift of 10 to this wave and the resultant signal is plotted below for comparison with the original signal. The solution can be verified by comparing the time-axis of both rectangular signals.



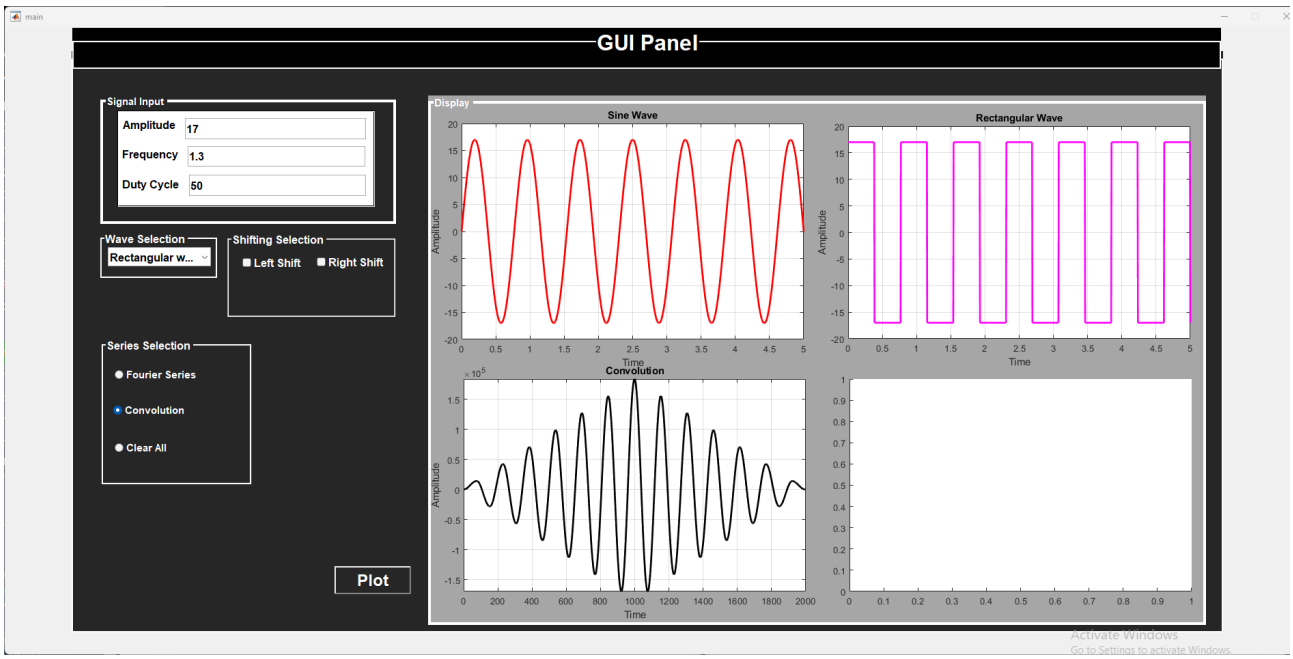
Screenshot 8 (Right-Shift)

Now I applied a Right Shift of 7 to this wave and the resultant signal is plotted below for comparison with the original signal. The solution can be verified by comparing the time-axis of both rectangular signals.



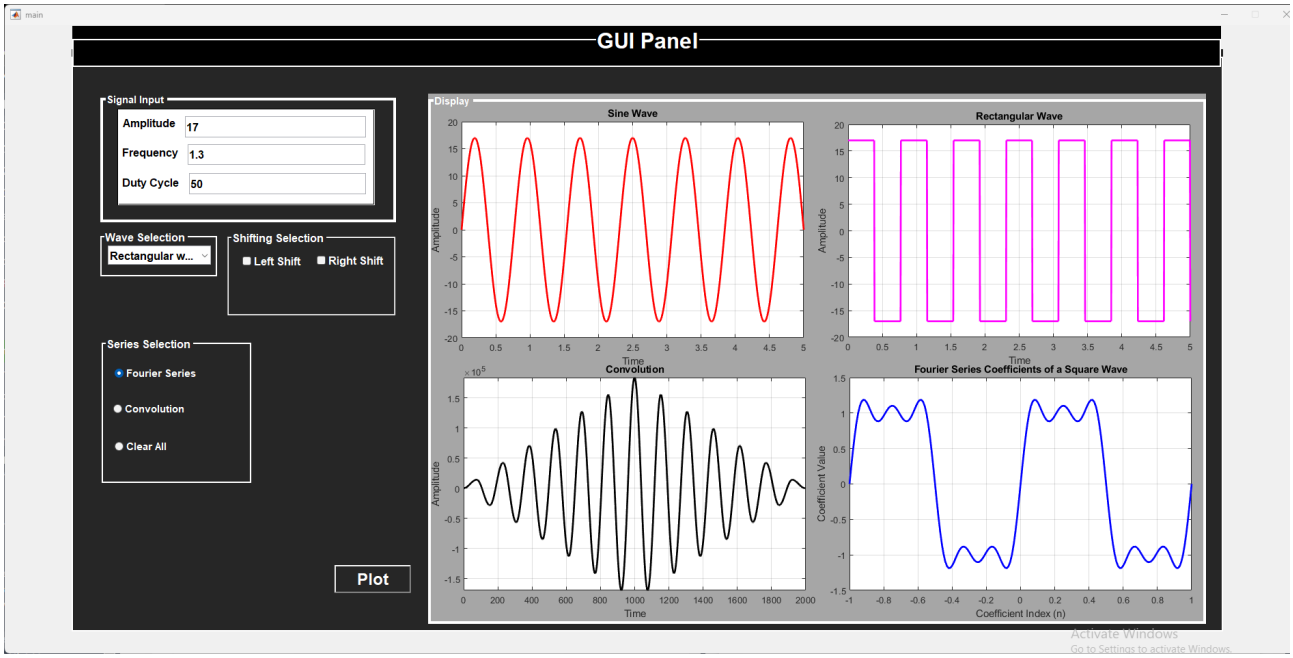
Screenshot 9 (Convolution)

Now I convoluted both the sin and rectangular wave together and plotted the resultant signal in third axes as shown in the figure below.



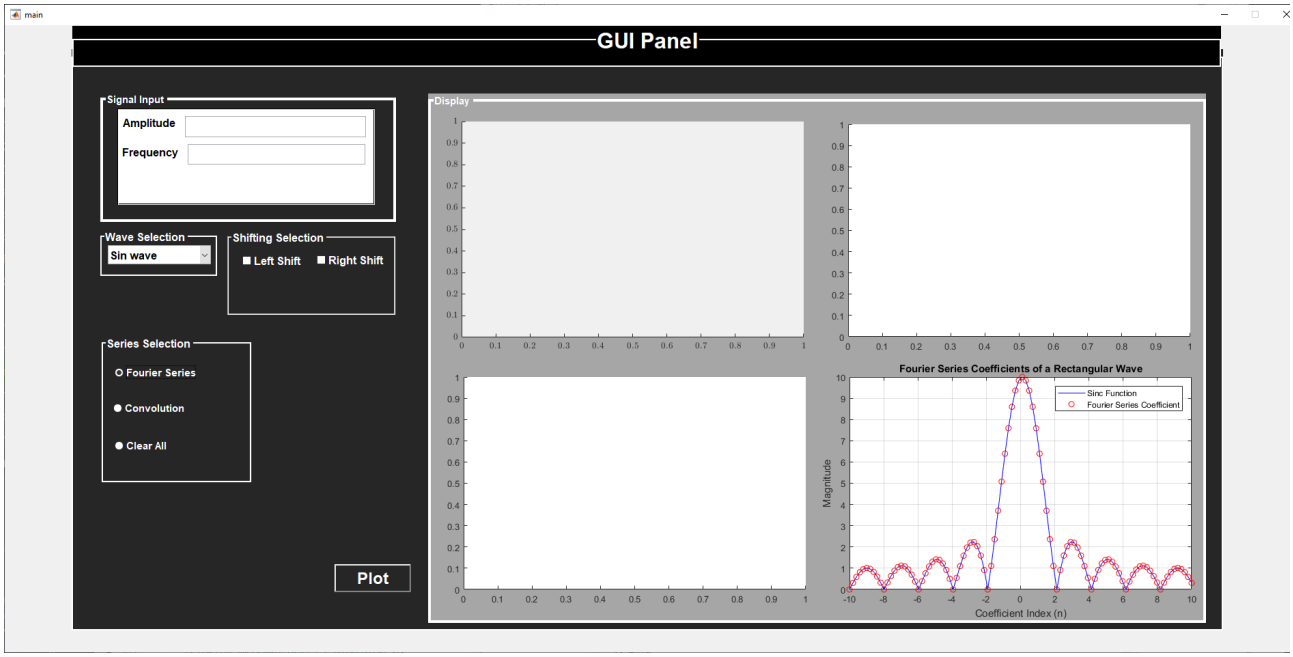
Screenshot 10 (Fourier Series coefficients)

Lastly, I plotted the Fourier Series coefficients for square ware in the 4th axes as shown below.



Screenshot 11 (Fourier Series coefficients)

After doing a deeper analysis of rectangular wave we obtain sinc function as the sum of Fourier Series coefficients where the blue line represents the sinc function and 'o' represents the harmonics or Fourier coefficients.



Since I plotted the abs values that's why are only able to visualize positive values.

7 Conclusion

In conclusion, the complex engineering problem of implementing a Graphical User Interface (GUI) using MATLAB for the course has been completed. The project involved setting up a User Interface with multiple options to obtain the desired Sin and Rectangular Wave.
The objective of this complex engineering activity is just to make an Interactive Signal Plotter, as we know there is always a need to plot the signals, so here in this CEA we are trying to make a User-Friendly GUI, where a person can come and interact with the GUI and can plot the signal according to his wish. Not only plotting but other functionalities can also be performed by using this GUI such as time shifting, convolution and Fourier series coefficient of a signal. Above mentioned features are all provided for CT Signals.